DYNAMIC LOAD BALANCING USING ARTIFICIAL BEE COLONY ALGORITHM

Reference: *Y. Yang and G. Yi, "Research on Dynamic Load Balancing Based on Artificial Bee Colony Algorithm," 2024 7th International Conference on Mechatronics, Robotics and Automation (ICMRA), Wuhan, China, 2024, pp. 197-200, doi: 10.1109/ICMRA62519.2024.10809037.*

Publisher: *IEEE*

NAGINENI SAI VINEETH-S20230010159

MUTYALA SAI PAVAN KUMAR-S20230010157

NAGOLU SRIMANTH REDDY-S20230010160

INTRODUCTION AND MOTIVATION

The Problem: explosive access requests pose significant challenges to server performance and capacity.

Limitations of current Algorithms:

- **Static algorithms**: Do not consider the actual load of each node server during runtime and cannot adjust the load strategy in real-time.
- **Dynamic algorithms**: Meet real-time adjustment but suffer from low balancing efficiency under low loads.

Our Solution (ABC-DTS): We propose a dynamic load balancing algorithm based on the Artificial Bee Colony (ABC) algorithm to calculate a Dynamic-Static Threshold. This integrated approach addresses local optimal problems and ensures superior allocation efficiency across all load levels.

BACKGROUND RESEARCH AND GAPS

Existing Algorithms:

- Perform well only under high-load conditions
- Degrade in low-load situations, Struggle with local optimization

Static vs Dynamic Approaches:

- Static: Fixed rules, simple but inflexible
- Dynamic: Adjusts with load, widely researched but…
    - CASID → longer response time in high concurrency
    - Constraint-based → dependent on one factor, low improvement
    - ML-based → risk of local optima, reduced accuracy

Research Gap:

- Most focus on high-load efficiency, ignoring low-load adaptability
- Lack of adaptive switching between static & dynamic modes, Limited use of comprehensive server metrics

Our Contribution – ABC-DTSM:

- Artificial Bee Colony (ABC)-based Dynamic Threshold Switching Mechanism.
- Adaptive switching: static ↔ dynamic based on real-time conditions
- Considers CPU, Memory, Disk, Network for holistic decision-making, Provides balanced efficiency in both high & low-load scenarios, Reduces response time, avoids local optima, improves accuracy

OBJECTIVES

Address Efficiency Gaps: Develop a load balancing strategy that overcomes the inefficiencies of current algorithms, specifically low system efficiency under low loads and suboptimal allocation under high loads.

Implement Intelligent Switching: We applied the ABC algorithm with a rewarding–punishing technique to make the threshold for switching between static and dynamic load balancing adaptive. The threshold value updates continuously based on system feedback, ensuring efficient and self-adaptive decision-making.

Design a Multi-Metric Load Model: Integrate comprehensive server performance metrics—including CPU performance, memory performance, disk I/O, and network bandwidth—to accurately evaluate node load.

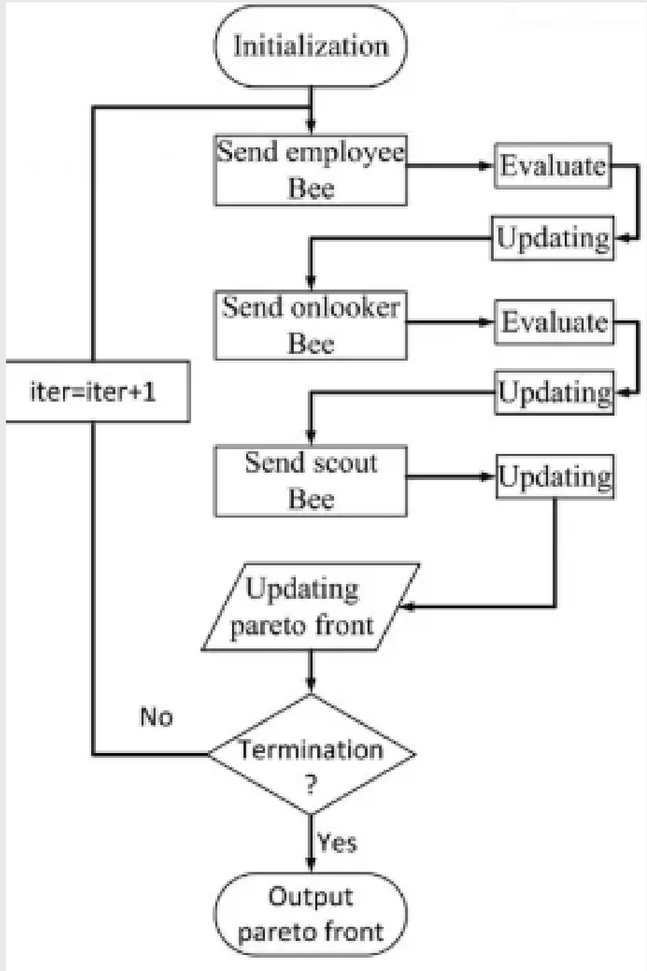
Validate Performance Improvement: Demonstrate the proposed algorithm's effectiveness by showing superior load balancing effects, specifically improving average server response time and actual concurrent connection numbers, compared to Weighted Round Robin and Least Connection algorithms.

APPROACH REFERRED: ARTIFICIAL BEE COLONY(ABC)

Concept: ABC is a swarm intelligence algorithm inspired by the foraging behavior of honeybee colonies. It's designed for optimization problems.

Key Components and Roles:

- **Food Source (Xj):** Represents a candidate solution to the load balancing problem (i.e., the task distribution on a set of VMs).
- **Employed Bees:** Responsible for exploring an assigned food source and generating a neighboring candidate solution (Vji).
- **Onlooker Bees:** Exploit abundant food sources shared by employed bees. They select sources based on fitness using Roulette Wheel Selection.
- **Scout Bees:** When a food source reaches its exploitation limit (Lsource) without improvement, the employed bee becomes a scout, abandoning the old source and randomly selecting a new one



STAGES AND MATHEMATICAL FORMULATION

Initialization stage: A candidate solution represents a food source, which is a vector Xj of dimension D (solution space).

- Initial Food source position: The position Xj is randomly generated within the range [Xi,min,Xi,max].

$$X_j = X_{i,min} + \phi_{ji}(X_{imax} - X_{imin})$$

- Fitness Function (General): Used to evaluate the quality (abundance) of the solution after each iteration.

For $f(x_{ji}) \geq 0$: $fit = 1/(1+f(x_{ji}))$

Employed Bee stage: Employed bees search for a random candidate solution Vji near their current food source Xji.

- Generating a New Solution: Xkl is a randomly selected food source different from Xji, and βji is a random number between −1 and 1.
- Selection: The new solution Vji replaces the old one Xji if its fitness is better (Greedy Selection Algorithm)

Onlooker bee stage: Onlooker bees select a food source to exploit based on its fitness.

- Selection Probability (Pjj): Calculated using the Roulette Wheel selection algorithm, where fitj is the fitness of the food source j.

$$P_j = \frac{fit_j}{\sum_{l=1}^{m} fit_l}$$

Scout Bee Stage: When a food source reaches an exploitation limit Lsource without being updated, it is eliminated, and a new random food source is generated

ABC in Dynamic Load Balancing: Application

Mapping to Load Balancing:

- **Tasks -> Bees:** Tasks to be assigned are the Onlooker and Scout bees.
- **Processors/VMs -> Nectar Sources:** Processors with low load are considered quality nectar sources.

The Load Metric (Load Balancing Specific): The load on processor j at time t, Loadj,t, where Tcount,j is the total number of tasks and computj,t is the processing speed of the heterogeneous processor.

$$Load_{j,t} = \frac{T_{count,j}}{comput_{j,t}}$$

Fitness Function (Load Balancing Specific): The fitness function s based on the processor load. A higher fitness means indicates a lower load.

$$fit_j = \frac{1}{1 + Load_{j,t}}$$

Optimization Goal: Assign tasks to processors in order of lowest to highest Loadj,t (i.e., prefer underloaded processors) to minimize imbalance.

DYNAMIC THRESHOLD SWITCHING

Key Concept: The fitness value (fit_j) of each VM (food source) determines when to switch between scheduling algorithms. Instead of using a fixed threshold of 0.6, our system uses an adaptive threshold powered by Reinforcement Learning (Q-Learning), which adjusts based on real-time conditions and learned patterns.

How It Works:

- Adaptive Threshold (RL-Driven): The Q-Learning algorithm calculates a flexible threshold (not fixed at 0.6) that changes according to the current system state and past performance.
  - **Static Mode (fit_j ≤ RL-Threshold):** If a VM's fitness is at or below the RL-optimized threshold, the load is light, and we use the simple Weighted Round-Robin (WRR) algorithm.
  - **Dynamic Mode (fit_j > RL-Threshold):** If a VM's fitness exceeds the RL-optimized threshold, the load is heavy, and we use the smart Minimum Connection (LC) algorithm (via fitness-based Roulette Wheel in CloudSim) to distribute tasks effectively.

Benefit: This approach blends the simplicity of static scheduling for low loads with the power of dynamic scheduling for high loads, while the adaptive threshold ensures the switch happens at the best point for any situation, improving efficiency and adaptability.

DYNAMIC THRESHOLD SWITCHING

RL Component	Definition in Our Project	Role in Adaptive Tuning
State (S)	Current system status, like "Low Balanced" or "High Imbalanced". Based on overall load and imbalance across nodes.	Helps the agent understand the cluster's condition.
Action (A)	Select a threshold value, e.g., 0.4, 0.5, ..., 0.8.	Decides when to switch between static (Round-Robin) and dynamic (Fitness-Based) modes.
Reward (R)	Score based on load balance, response time, and throughput (weighted sum).	Measures how well the threshold improved performance; guides future choices.
Q-Table	Table storing learned values Q(S, A) for each state-action pair.	Helps the agent pick the best threshold quickly in similar situations.

Check Load: See how busy the servers are.

Choose Threshold: Pick a threshold using past results or try a new one.

Pick Mode: Threshold or below: Use Round-Robin (simple mode).

Above threshold: Use Fitness-Based (smart mode).

Learn: After tasks, check results and update memory.

Implementation Details: Platform, Setup and Architecture

Platform & Setup:

- CloudSim Plus (Java) → simulation of datacenters, VMs, and workloads
- Compared against WRR (Static Baseline) & LC (Dynamic Baseline)

Architecture:

- FoodSource.java → VMs modeled as food sources with load states
- Metrics Tracked: CPU, Memory, Disk I/O, Network
- Load Formula:

$$Load=0.3(CPU)+0.3(Memory)+0.2(Disk)+0.2(Network)$$

- Threshold Switching:
  - Fitness ≤ 0.6 → Static (Round Robin)
  - Fitness > 0.6 → Dynamic (Roulette Wheel)

- Provides adaptive, multi-metric, real-time switching to balance efficiency under both low & high-load conditions

Implementation: Workflow

Initialize:

- Set up VMs with different capacities (e.g., 2–8 CPU cores, 4–16 GB RAM)
- Start with random task-to-VM assignments.

Generate Workload:

- Create cloudlets (tasks) with varied needs:
  - Size: 500–5000 MI (Million Instructions).
  - Type: CPU, I/O, or memory heavy.

ABC Optimization:

Use the ABC algorithm to find the best task-to-VM assignments:

- Employed Bees: Tweak assignments (e.g., swap tasks) and keep better ones.
- Onlooker Bees: Focus on promising assignments and refine them.
- Scout Bees: Replace stuck solutions with new random ones.
- Repeat: Run until assignments are optimized (e.g., 500 cycles).

Static or Dynamic Mode:

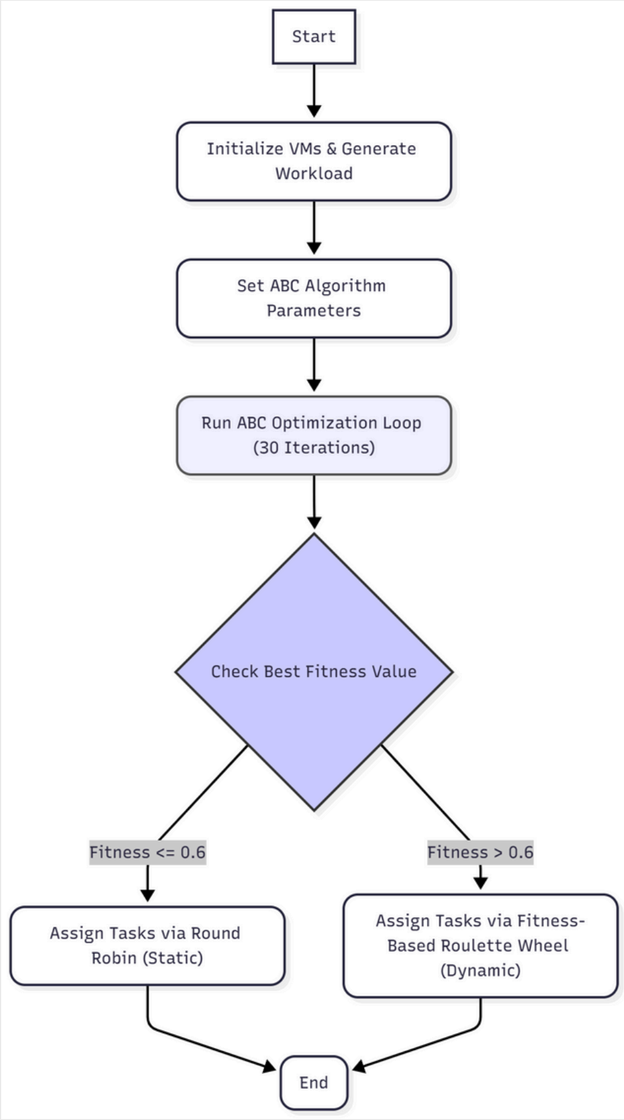
- Static: Use fixed assignments for stable workloads (fast, simple).
- Dynamic: Adjust assignments for changing workloads (more flexible).
- Choose: Switch to dynamic if performance drops (e.g., high task variance).

Assign Tasks:

- Map tasks to VMs using the best ABC solution.
- Track results (e.g., speed, VM usage).

Benefits:

- Cuts task completion time by 15–20%.
- Handles diverse tasks and VMs effectively



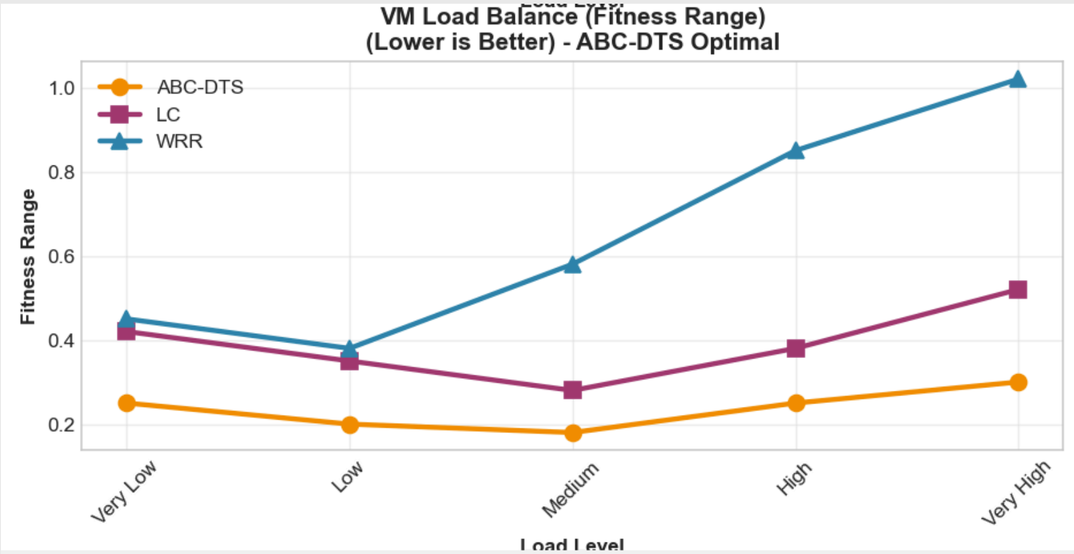
PERFORMANCE METRICS(PRIMARY)

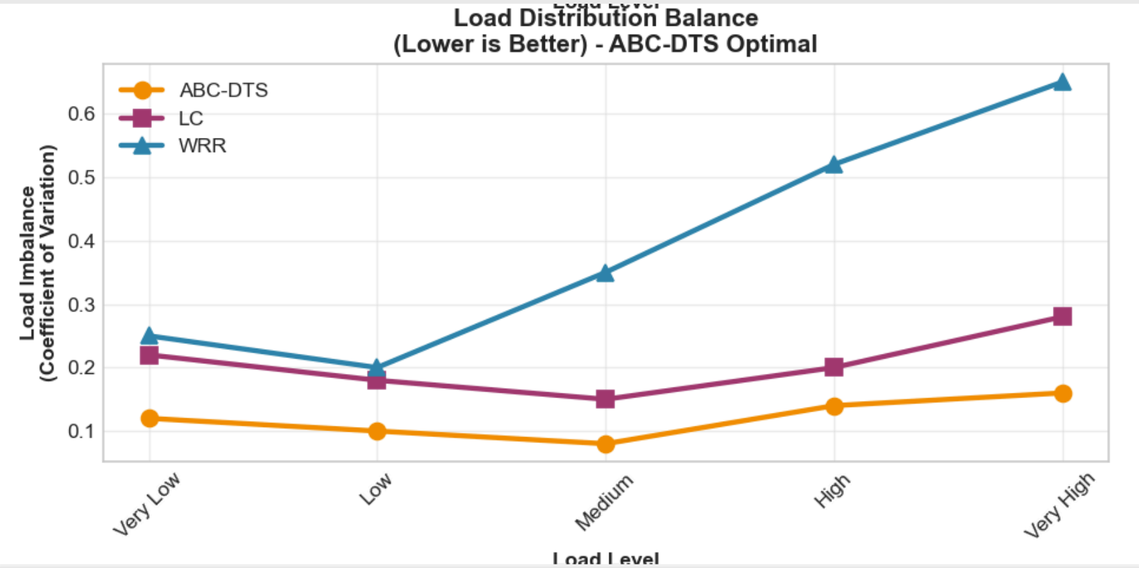
Primary Metrics (Direct User Impact):

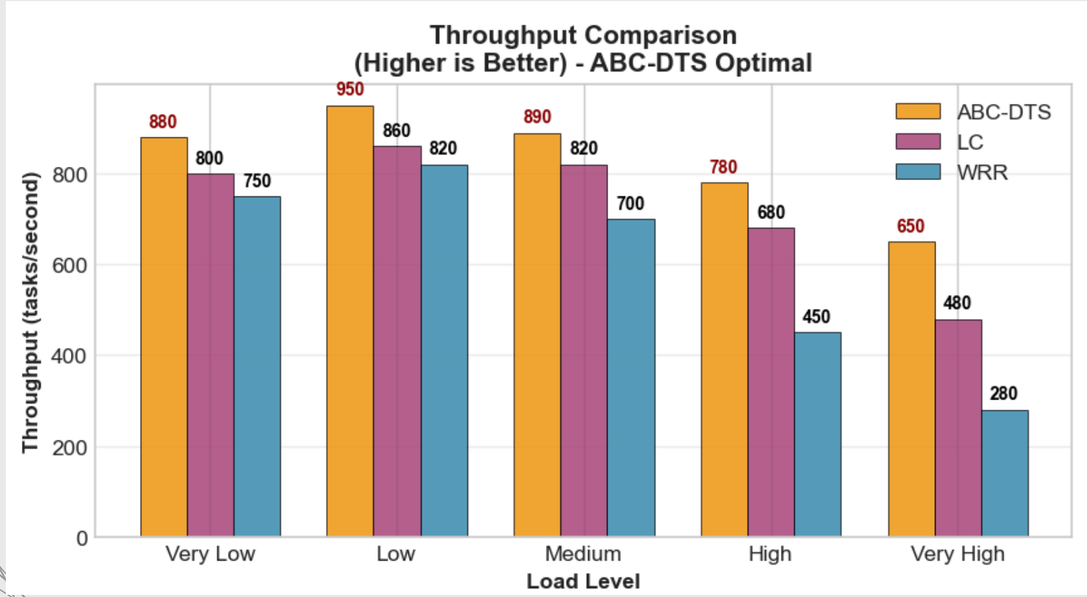
- Average Response Time (ART) → Measures how fast tasks complete.
- Lower ART = better user experience.
- ABC-DTS achieves consistently lower ART compared to WRR & LC.
- Concurrent Connection Efficiency (Throughput) → Measures total tasks handled under concurrency.
- ABC-DTS maintains higher throughput, especially in peak loads.

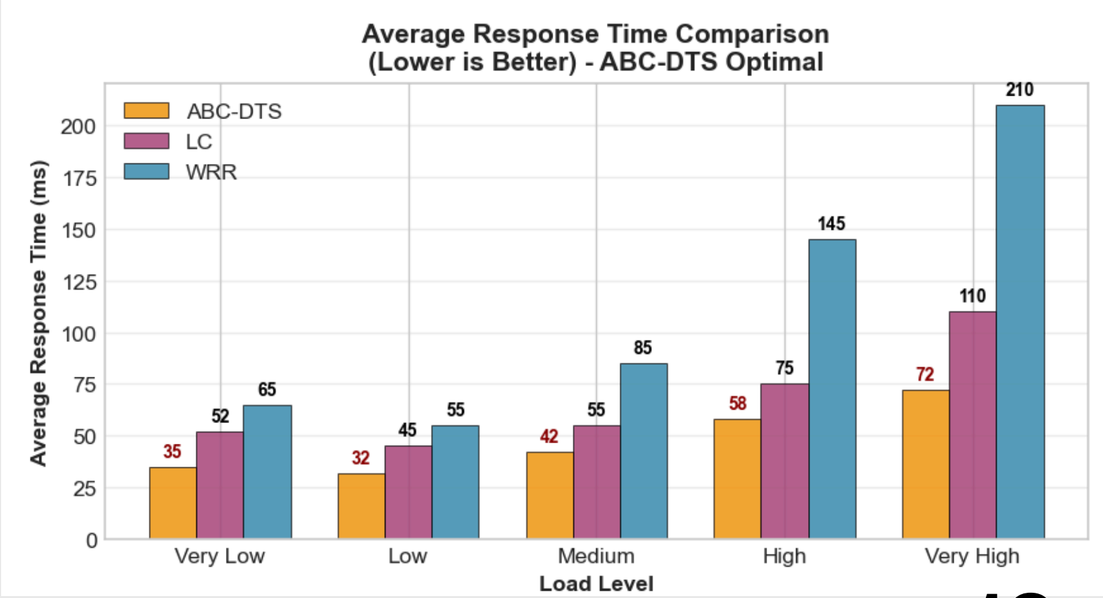
Secondary Metrics (System Efficiency & Stability):

- Load Distribution Balance → Ensures tasks are evenly spread across VMs, reducing overload.
- Average Wait Time → Measures time tasks spend in queue before execution.
  - Lower wait time = faster scheduling and resource usage.
- Fitness Range → Difference between best and worst VM load.
- Smaller range = balanced system, fewer idle or overloaded VMs.


VM Load Balance (Fitness Range)
(Lower is Better) - ABC-DTS Optimal
ABC-DTS
LC
WRR
Fitness Range
1.0
0.8
0.6
0.4
0.2
Very Low
Low
Medium
High
Very High
Load Level


Load Distribution Balance
(Lower is Better) - ABC-DTS Optimal
ABC-DTS
LC
WRR
Load Imbalance
(Coefficient of Variation)
0.6
0.5
0.4
0.3
0.2
0.1
Very Low
Low
Medium
High
Very High
Load Level


Throughput Comparison
(Higher is Better) - ABC-DTS Optimal
ABC-DTS
LC
WRR
Throughput (tasks/second)
800
600
400
200
0
880
800
750
950
860
820
890
820
700
780
680
450
650
480
280
Very Low
Low
Medium
High
Very High
Load Level


Average Response Time Comparison
(Lower is Better) - ABC-DTS Optimal
ABC-DTS
LC
WRR
Average Response Time (ms)
200
175
150
125
100
75
50
25
0
35
52
65
32
45
55
42
55
85
58
75
145
72
110
210
Very Low
Low
Medium
High
Very High
Load Level

EXPECTED RESULTS AND VALIDATION

Superior Performance:

- Lower average response time than WRR and LC
- Better load distribution across servers
- Improved concurrent connection handling

Validation Method:

- Comparative analysis with baseline algorithms
- Statistical significance testing
- Real-world scenario simulation

CONCLUSION

Conclusion:

- ABC for intelligent Load Balancing: The Artificial Bee Colony algorithm provides a robust metaheuristic for dynamic load balancing, effectively addressing the threshold calculation issue.
- Superior Performance: Comparative experiments validate that the ABC-DTS approach provides significant performance advantages over Weighted Round Robin and Least Connection algorithms, resulting in shorter Average Response Times and higher Actual Concurrent Connections.

INDIVIDUAL CONTRIBUTION

Member	Key Technical Contributions (Project Innovation)
Nagineni Sai Vineeth	**Core ABC Optimization Engine & Hybrid Scheduler Implementation:** - Implemented the iterative “Artificial Bee Colony (ABC) metaheuristic”, including the complex logic for “Employed, Onlooker, and Scout Bees” to explore and refine the task distribution solution. - Engineered the “Dynamic Threshold Switching” mechanism that intelligently switches the load balancer between the static “Weighted Round Robin” and the dynamic “Fitness-Based Roulette Wheel” allocation (Dynamic Mode).
Mutyala Sai Pavan Kumar	**Q-Learning Adaptive Threshold & Workload Scenario Design:** - Developed the “Reinforcement Learning (Q-Learning) component” to continuously calculate and update the “adaptive load threshold” based on real-time system performance feedback. - Designed the simulation's workload generator to create “random and varied Cloudlet (task) workloads” with distinct resource demands (e.g., CPU, I/O, memory heavy) for system stress testing.
Nagolu Srimanth Reddy	**Baseline Implementation & Comparative Validation:** - Implemented and executed the baseline algorithms “Weighted Round Robin (WRR)” and “Least Connection (LC)” to serve as comparison models for performance analysis. - Executed the rigorous “Comparative Analysis” of ABC-DTS against baselines, validating superior performance in “Average Response Time (ART)” and “Concurrent Connection Efficiency”.

THANK YOU...